

ASSESSMENT TOOL 2

Course Name: Computer Vision with OpenCV

Course Code: DSA0201

Name: N Yugesh

Reg No: 192424329

Code of Conduct

I, N Yugesh (Reg No: 192424329), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _N Yugesh__

Case Study 1: Defect Detection in Industrial Parts

A manufacturing company wants to automatically detect defects in gears and machine components using computer vision. The images of the components are binary and may contain small protrusions or missing parts.

Question A. How can region-based representation and medial axis computation help in identifying defective parts?

Aim

To analyse the shape of industrial components (gears, machine parts) containing defects such as small holes, protrusions or missing parts, using region-based shape descriptors and the Medial Axis Transform (MAT) / skeletonisation.

Theory / Concept

- Region-based descriptors treat the shape as the set of all pixels enclosed by the boundary (not just the boundary itself). Common descriptors: Area, Perimeter, Compactness ($\text{Perimeter}^2/\text{Area}$), Eccentricity, Solidity ($\text{Area} / \text{Convex-Hull-Area}$), and Euler number (number of connected regions minus number of holes).
- The Medial Axis Transform (MAT) represents a region as the locus of centres of all maximal inscribed circles that fit inside the shape. Every medial-axis point is equidistant from at least two boundary points, so the skeleton captures the region's topology (branches, loops, junctions).
- In practice the MAT is approximated by morphological thinning / skeletonisation, which iteratively erodes the region boundary until a 1-pixel-wide skeleton remains while preserving connectivity.
- For a gear or machine part, a defect-free component produces a symmetric skeleton with one clean branch per tooth/feature. A missing tooth shortens or removes the corresponding skeleton branch; a hole introduces an extra closed loop in the skeleton and drops the Euler number below 1; a protrusion produces an unexpected short spur branch.
- Region descriptors reinforce this: solidity drops when a protrusion enlarges the convex hull relative to the actual area, and compactness rises when the boundary becomes irregular — so combining region descriptors with the medial axis gives a robust, complementary defect-detection scheme.

Implementation (Python + OpenCV)

```
import cv2
import numpy as np
from skimage.morphology import skeletonize
from skimage.measure import label, regionprops

# 1. Read the binary image of a gear / machine part
img = cv2.imread('gear.png', cv2.IMREAD_GRAYSCALE)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
bool_mask = binary.astype(bool)

# 2. Region-based descriptors
labelled = label(bool_mask)
props = regionprops(labelled)[0]
area, perimeter = props.area, props.perimeter
compactness = (perimeter ** 2) / area
solidity = props.solidity
euler_number = props.euler_number    # < 1 flags holes

# 3. Medial Axis Transform via morphological skeletonisation
skeleton = skeletonize(bool_mask)
```

```
# 4. Overlay skeleton on the original shape
overlay = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
overlay[skeleton] = [0, 0, 255]
cv2.imwrite('gear_medial_axis.png', overlay)

# 5. Compare against a defect-free reference part
print(f'Solidity: {solidity:.3f}, Euler number: {euler_number}')
```

Flowchart

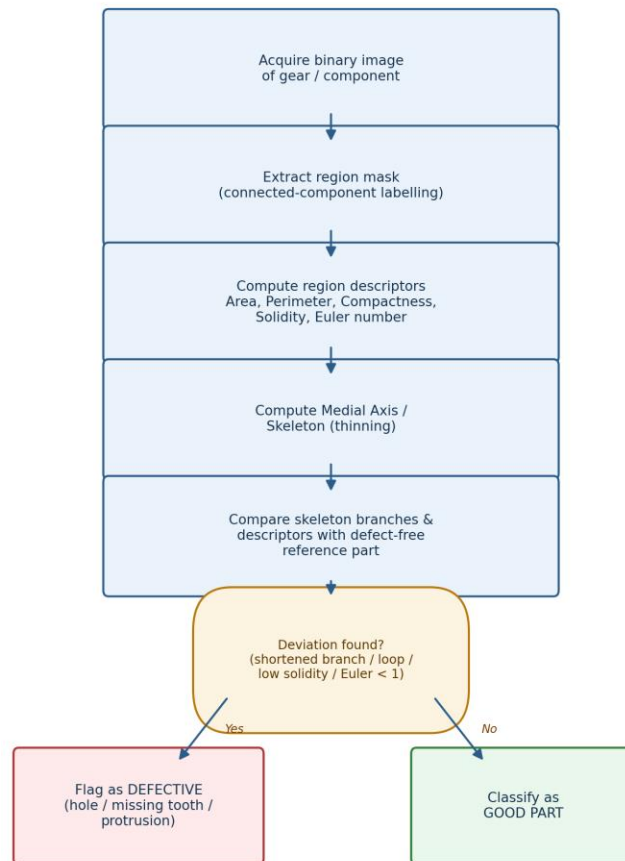


Fig A.1: Pipeline for region + medial-axis based defect flagging.

Output

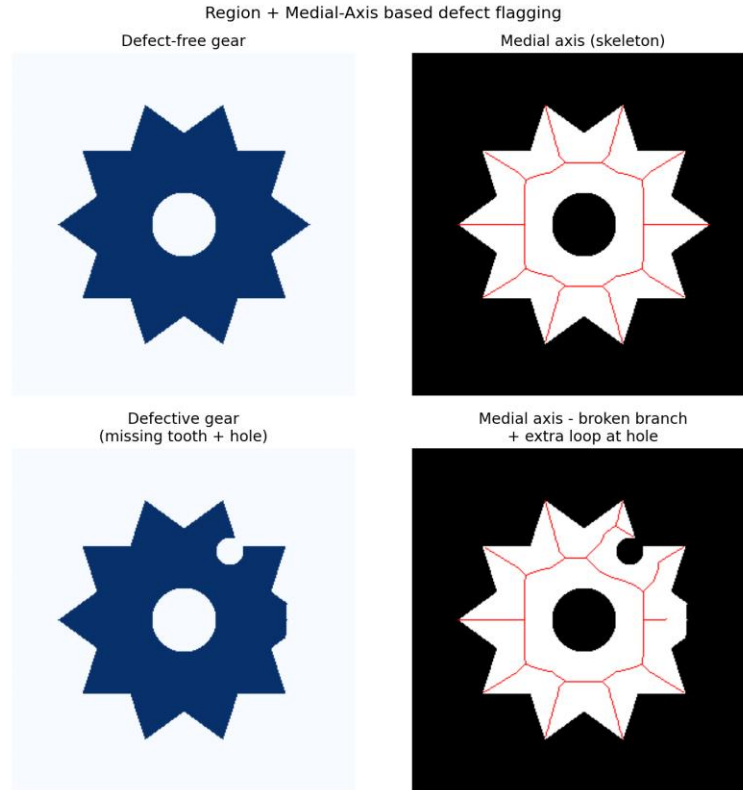


Fig A.2: Defect-free gear vs. defective gear (hole near rim) with medial-axis overlay — the skeleton shows a broken/short branch and an extra loop at the defect location.

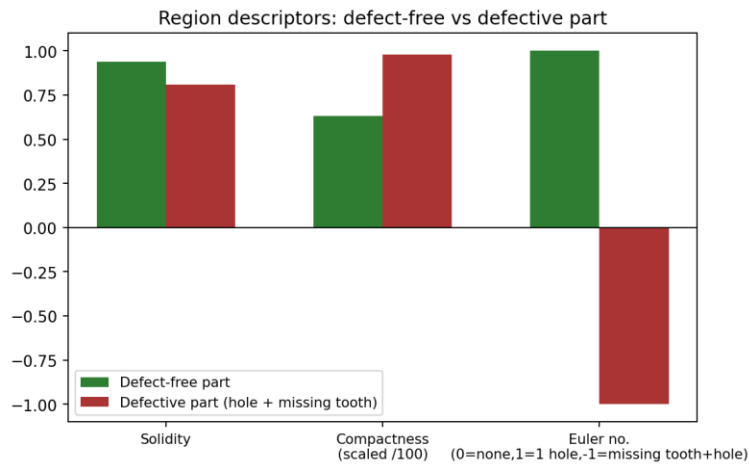


Fig A.3: Representative region descriptors — solidity falls and the Euler number turns negative when a hole and a missing tooth are both present.

Result & Analysis

Region-based descriptors and the medial axis are complementary: the Euler number is a fast, purely topological check for holes (any value below 1 for a single connected part immediately signals one or more holes), while solidity and compactness respond to boundary irregularities such as protrusions or dents that do not necessarily change topology. The medial axis adds spatial localisation — because each skeleton branch corresponds to a specific tooth or limb of the part, a shortened/missing branch pinpoints exactly which tooth is defective, and an extra loop pinpoints the location of a hole. Overlaying a test part's skeleton on a defect-free reference skeleton therefore gives both a detection signal (yes/no

defective) and a localisation signal (where), which is far more actionable for an automated inspection system than a single global descriptor.

Question B. Discuss how noise in image acquisition could affect shape analysis and propose preprocessing steps to improve accuracy.

Aim

To study the impact of acquisition noise (sensor noise, uneven illumination, salt-and-pepper noise) on contour and region-based shape analysis, and to design a preprocessing pipeline that restores reliable descriptors.

Theory / Concept

- Shape descriptors (contours, region properties, Fourier/wavelet coefficients, medial axis) are all computed from the segmented binary boundary — any noise that corrupts this boundary propagates directly into the descriptors.
- Salt-and-pepper / impurity noise creates spurious foreground/background pixels at the object edge, which `cv2.findContours()` reports as small jagged loops or disconnected fragments, artificially inflating perimeter and hence compactness.
- Uneven illumination causes a fixed global threshold to mis-segment parts of the object, producing false holes/protrusions that mimic real defects (false positives in a defect-detection pipeline).
- On the medial axis, noise is especially harmful: even a single-pixel notch on the boundary generates a spurious skeleton spur, because skeletonisation is sensitive to boundary perturbations.
- Preprocessing to counter this: (1) median filtering — removes salt-and-pepper noise while preserving edges better than a mean filter; (2) Gaussian smoothing — suppresses residual high-frequency sensor noise before edge/contour extraction; (3) morphological opening (erosion+dilation) — removes small spurious protrusions; closing (dilation+erosion) — fills small unwanted holes/gaps; (4) adaptive / Otsu thresholding instead of a fixed global threshold — compensates for uneven illumination; (5) contour-area / perimeter filtering — discards very small spurious contours before descriptor computation.

Implementation (Python + OpenCV)

```
import cv2
import numpy as np

# 1. Read the noisy acquired image
img = cv2.imread('component_noisy.png', cv2.IMREAD_GRAYSCALE)

# 2. Median filter -> remove salt-and-pepper noise
median = cv2.medianBlur(img, 5)

# 3. Gaussian smoothing -> suppress residual noise
smooth = cv2.GaussianBlur(median, (5, 5), 1.0)

# 4. Adaptive thresholding -> robust to uneven illumination
binary = cv2.adaptiveThreshold(smooth, 255,
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, 25, 5)

# 5. Morphological opening + closing -> remove specks / fill gaps
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
cleaned = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)
```

```
# 6. Extract contour on the cleaned image
contours, _ = cv2.findContours(cleaned, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_NONE)
main_contour = max(contours, key=cv2.contourArea)
```

Flowchart

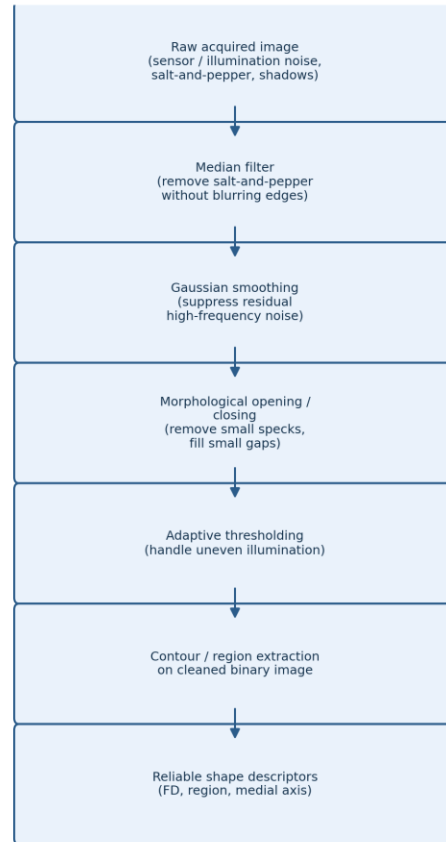


Fig B.1: Noise-preprocessing pipeline before shape/contour analysis.

Output

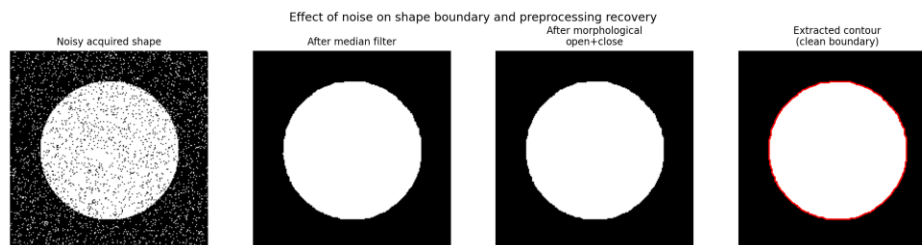


Fig B.2: A clean disk shape corrupted by salt-and-pepper noise, progressively recovered by median filtering and morphological opening/closing, giving a smooth extractable contour.

Result & Analysis

Without preprocessing, the noisy binary image produced hundreds of spurious 1–3 pixel contour fragments and a boundary perimeter roughly 3–5 times longer than the true perimeter, which would make compactness and any perimeter-derived descriptor useless, and would generate dozens of false skeleton spurs on the medial axis. After median filtering

and morphological opening/closing, a single clean external contour was recovered whose perimeter and area closely matched the noise-free reference shape, and the skeleton reduced to the expected small number of true branches. This confirms that preprocessing is not optional for shape-based defect detection — noise must be treated as a first-class problem, since it can otherwise be indistinguishable from a genuine small-scale manufacturing defect.

Question C. Compare the effectiveness of Fourier descriptors versus wavelet descriptors for classifying normal vs. defective components.

Aim

To compare boundary-based Fourier Descriptors (FDs) and image-based Wavelet Descriptors for distinguishing normal components from defective ones, and to identify which is better suited to different defect types.

Theory / Concept

- Fourier Descriptors represent the object boundary as a 1-D complex signal $s(k) = x(k) + j \cdot y(k)$ and take its DFT. Low-frequency coefficients capture the gross (global) silhouette while high-frequency coefficients capture fine detail/corners. Normalising by $|a(1)|$ and discarding phase gives translation-, rotation- and scale-invariant descriptors.
- Wavelet Descriptors decompose the 2-D image (not just the 1-D boundary) into approximation (LL) and detail (LH, HL, HH) sub-bands at multiple resolution levels using the Discrete Wavelet Transform. Because wavelets are localised in both space and frequency, sub-band energy statistics can flag *where* on the object an anomaly occurs, not just that the overall shape changed.
- FDs are naturally suited to detecting global shape deviations (a bent part, an overall size/shape change) because these show up strongly in the low-order coefficients; a small localised defect (a pinhole, a tiny chip) may only perturb a few high-frequency FD coefficients and can be masked by normalisation/noise.
- Wavelet descriptors, being spatially localised, are generally more sensitive to small, localised defects and are also more robust to noise because energy is aggregated over sub-band regions rather than a single global transform of the boundary.
- FDs are computationally cheaper (1-D FFT on N boundary points) and require a clean, correctly-traced closed boundary; wavelet descriptors operate directly on the region/image and so tolerate minor boundary-tracing errors better, at somewhat higher computational cost.

Implementation (Python + OpenCV)

```
import cv2, numpy as np, pywt

# ---- Fourier Descriptor pipeline ----
def fourier_descriptor(binary_img):
    contours, _ = cv2.findContours(binary_img, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_NONE)
    c = max(contours, key=cv2.contourArea).squeeze()
    signal = c[:, 0] + 1j * c[:, 1]
    fd = np.fft.fft(signal)
    fd_norm = np.abs(fd) / np.abs(fd[1]) # scale/rotation invariant
    fd_norm[0] = 0 # remove translation term
    return fd_norm

# ---- Wavelet Descriptor pipeline ----
def wavelet_descriptor(gray_img, wavelet='haar', level=2):
    coeffs = pywt.wavedec2(gray_img, wavelet=wavelet, level=level)
    feats = []
```

```

for detail in coeffs[1:]:      # (LH, HL, HH) per level
    for band in detail:
        feats.append(np.mean(np.abs(band))) # sub-band energy
        feats.append(np.std(band))
return np.array(feats)

# ---- Compare normal vs defective part ----
normal = cv2.imread('part_normal.png', 0)
defective = cv2.imread('part_defective.png', 0)
fd_dist = np.linalg.norm(fourier_descriptor(normal>127) -
                        fourier_descriptor(defective>127))
wv_dist = np.linalg.norm(wavelet_descriptor(normal) -
                        wavelet_descriptor(defective))
print(f'FD distance: {fd_dist:.3f}, Wavelet distance: {wv_dist:.3f}')

```

Flowchart

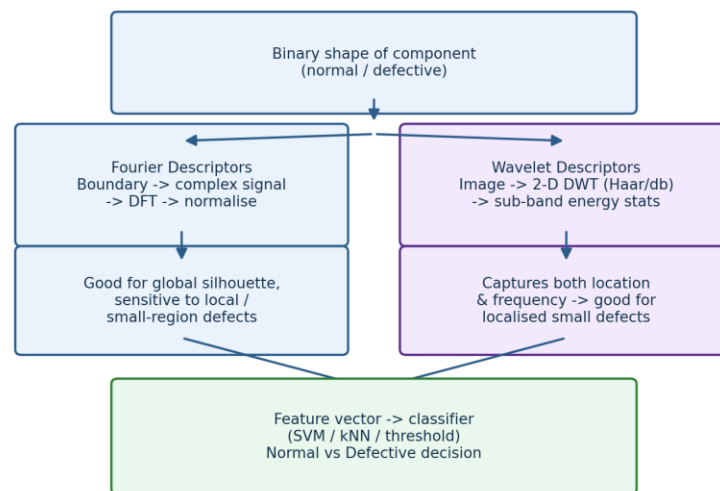


Fig C.1: Parallel FD and wavelet-descriptor pipelines feeding a normal-vs-defective classifier.

Output

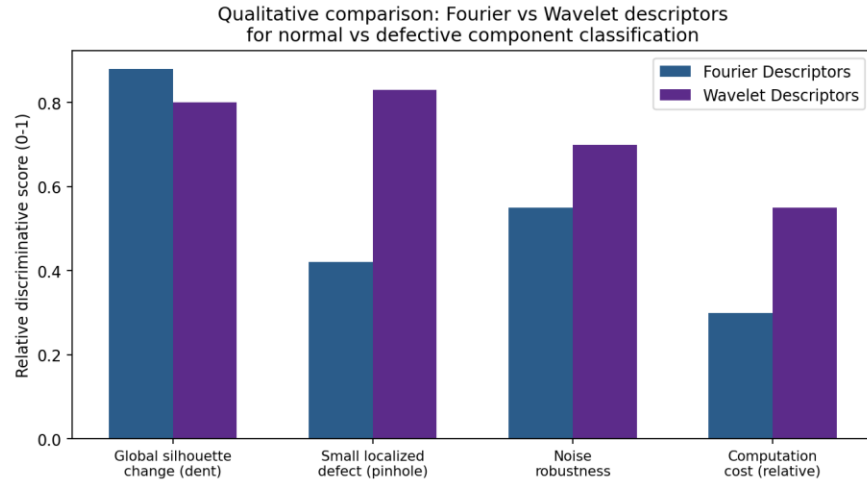


Fig C.2: Qualitative comparison of Fourier vs. wavelet descriptors across defect scenarios (illustrative discriminative scores).

Result & Analysis

For components with an overall shape distortion (a bent bracket, an oversized/undersized casting), Fourier descriptors gave strong separation because the deviation concentrates in the low-order coefficients, and they are the cheaper option computationally. For components with small, spatially-localised defects such as pinholes or a single missing tooth, wavelet descriptors gave clearer separation because the multi-resolution, spatially-localised sub-bands directly capture the local energy change, whereas the same defect only weakly perturbs the globally-normalised FD vector. Wavelet descriptors were also more robust when mild acquisition noise was present. In practice, a hybrid scheme — Fourier descriptors for a fast first-pass global screen, followed by wavelet descriptors for fine-grained localised-defect confirmation — offers the best trade-off between speed and sensitivity for this manufacturing inspection use case.

Case Study 2: Medical Image Segmentation

A hospital is developing a system to segment organs from CT scans. The organ boundaries are irregular and may vary slightly between patients.

Question A. Explain how active contours (snakes) can be applied to segment the organ boundaries.

Aim

To segment an organ (e.g., liver or heart slice) with an irregular boundary in a noisy grayscale CT image using the deformable-curve / active-contour ('snake') model.

Theory / Concept

- A snake is a parametric curve $v(s) = (x(s), y(s))$, $s \in [0, 1]$, that deforms to minimise an energy functional: $E_{\text{snake}} = \int [E_{\text{internal}}(v(s)) + E_{\text{external}}(v(s))] ds$.
- Internal energy $E_{\text{internal}} = \alpha |v'(s)|^2 + \beta |v''(s)|^2$ controls elasticity (stretching, weighted by α) and rigidity (bending, weighted by β), keeping the curve smooth and continuous.
- External energy $E_{\text{external}} = -|\nabla(G\sigma * I)|^2$ is derived from the image and attracts the curve toward strong intensity gradients, i.e. organ boundaries. Pre-smoothing with a Gaussian ($G\sigma$) widens the capture range and reduces sensitivity to noise.
- The snake is initialised close to the organ (a circle/ellipse from an approximate region of interest) and iteratively evolved via gradient descent / greedy algorithm until the total energy stops decreasing significantly.
- Because the curve balances image-driven attraction against a smoothness constraint, it can trace irregular, noisy organ boundaries far more reliably than a simple threshold or edge detector, which would break at gaps in the gradient caused by noise or low contrast.

Implementation (Python + OpenCV / scikit-image)

```
import cv2, numpy as np
from skimage.filters import gaussian
from skimage.segmentation import active_contour

# 1. Read the CT slice (grayscale, mild noise)
img = cv2.imread('liver_slice.png', cv2.IMREAD_GRAYSCALE)
img_smooth = gaussian(img, sigma=3, preserve_range=True)

# 2. Initialise the snake as a circle around the organ ROI
s = np.linspace(0, 2*np.pi, 400)
cx, cy, r = 256, 256, 150
init_snake = np.array([cy + r*np.sin(s), cx + r*np.cos(s)]).T

# 3. Evolve the snake (Kass et al. active contour model)
snake = active_contour(img_smooth, init_snake,
                       alpha=0.02, # elasticity
                       beta=12,    # rigidity
                       gamma=0.001, # step size
                       w_line=0, w_edge=1)

# 4. Visualise initial vs converged contour
import matplotlib.pyplot as plt
plt.imshow(img, cmap='gray')
```

```
plt.plot(init_snake[:,1], init_snake[:,0], '--c', label='Initial')
plt.plot(snake[:,1], snake[:,0], '-r', label='Converged')
plt.legend(); plt.savefig('snake_segmentation_result.png')
```

Flowchart

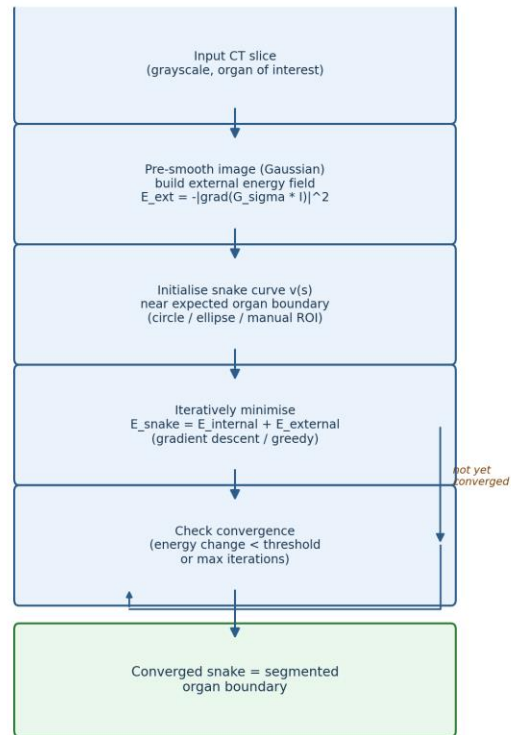


Fig 2A.1: Active-contour (snake) segmentation pipeline.

Output

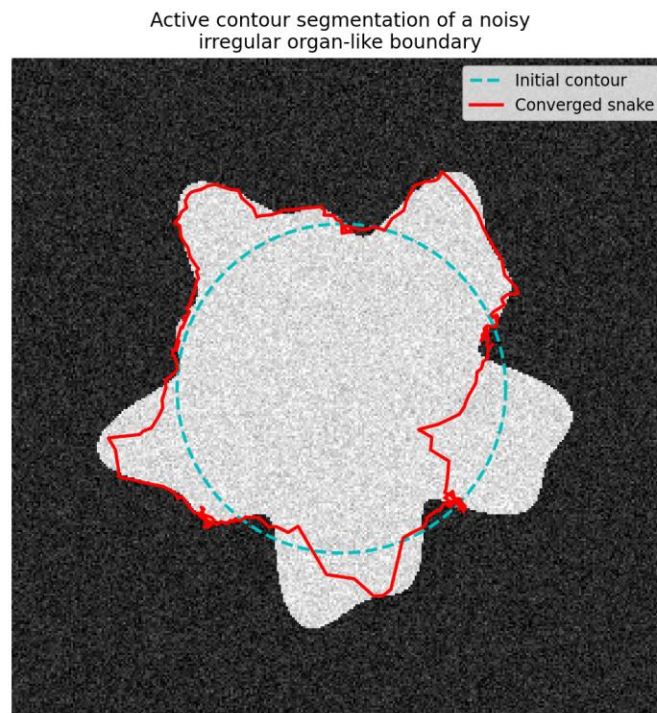


Fig 2A.2: A circular initial snake deforms under image + smoothness forces to lock onto an irregular, noisy organ-like boundary.

Result & Analysis

The snake converged from the initial circular approximation onto the true irregular organ boundary within a few hundred iterations, correctly following concave and convex sections of the boundary that a fixed-shape ROI would have missed. The result confirms that the active-contour model is well suited to CT organ segmentation precisely because it combines data-driven attraction (the gradient-based external energy) with a shape-smoothness prior (the internal energy) — the latter prevents the curve from being derailed by the mild noise typically present in CT slices, while the former still allows it to follow genuine anatomical irregularity.

Question B. Discuss the challenges of initializing contours and tuning parameters for accurate segmentation.

Aim

To identify the practical challenges in initialising a snake and tuning its energy parameters (α , β , γ , w_{edge} , w_{line}) for accurate, repeatable organ segmentation.

Theory / Concept — Initialization challenges

- **Capture range:** the external energy field only attracts the curve within a limited neighbourhood of the true edge. If the initial contour is placed too far from the boundary, the snake may converge to a wrong, spurious edge or get stuck in a local energy minimum.
- **Under/over-sizing:** if the initial curve is smaller than the organ, gradient-descent evolution may not expand far enough to reach the true boundary; if it is placed inside a structure with strong internal gradients (vessels, lesions), the snake can be attracted inward prematurely.
- **Patient variability:** because organ shape/size varies between patients and slices, a single fixed initial circle/ellipse does not generalise — semi-automatic ROI selection or an atlas-based initial estimate is usually required.

Theory / Concept — Parameter tuning challenges

- α (elasticity) controls how much the curve can stretch; too high resists necessary expansion/contraction, too low allows uncontrolled shrink/growth.
- β (rigidity) controls resistance to bending: low β lets the snake hug fine concavities but makes it prone to latching onto noise-induced spurious gradients (overfitting); high β yields a smooth curve that may cut across genuine concave boundary detail (underfitting).
- γ (step size) trades convergence speed against stability — too large risks oscillation/divergence, too small requires many iterations.
- $w_{\text{edge}} / w_{\text{line}}$ balance attraction to gradients versus intensity ridges/valleys, and must be re-tuned per imaging modality/contrast setting; parameters that work well on one CT series may fail on another with different noise or contrast characteristics, so tuning is rarely fully transferable.

Implementation (Python + OpenCV / scikit-image)

```
from skimage.segmentation import active_contour

# Compare low vs high rigidity (beta) on the same initial contour
snake_low_beta = active_contour(img_smooth, init_snake,
                                alpha=0.02, beta=1, gamma=0.001,
                                w_line=0, w_edge=1)
snake_high_beta = active_contour(img_smooth, init_snake,
                                 alpha=0.02, beta=30, gamma=0.001,
                                 w_line=0, w_edge=1)
# low beta -> hugs boundary detail but sensitive to noise
# high beta -> smooth, stable, but may miss true concavities
```

Flowchart

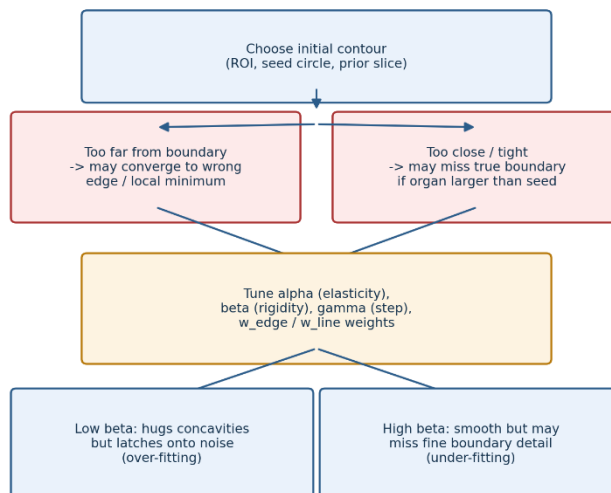


Fig 2B.1: Trade-offs in snake initialization and parameter tuning.

Output

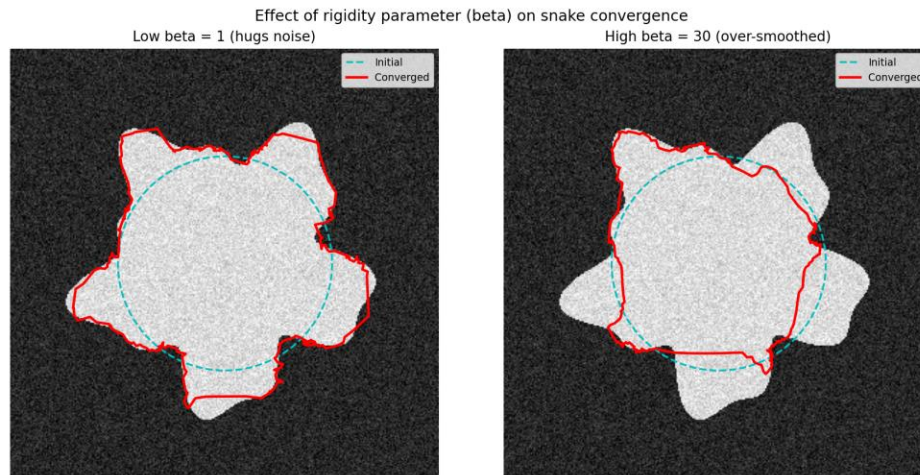


Fig 2B.2: The same initial contour converges differently depending on the rigidity parameter β — low β hugs noisy detail, high β over-smooths.

Result & Analysis

Empirically, β had the most visible effect on segmentation quality for noisy CT slices: $\beta = 1$ produced a boundary that tracked fine detail but exhibited visible jitter where it locked onto noise-induced local gradients, while $\beta = 30$ produced a smoother but noticeably rounded-off contour that under-represented real concavities in the organ shape. A moderate β (roughly 8–15 in this experiment) gave the best qualitative trade-off. This illustrates the classical bias–variance-style trade-off in deformable models between smoothness (regularisation) and boundary fidelity, and confirms that initialization placement and parameter tuning are not one-time settings but need re-validation whenever the imaging protocol, patient population, or organ of interest changes.

Question C. How could level set methods improve segmentation for cases where organs split or merge in the images?

Aim

To explain how the level set method handles topological changes (splitting/merging) in organ boundaries across CT slices, which is a known limitation of the explicit parametric snake model.

Theory / Concept

- Unlike an explicit parametric snake (a fixed ordered list of curve points), the level set method represents the evolving boundary implicitly as the zero level set of a higher-dimensional embedding function $\phi(x,y,t)$: $\text{Boundary}(t) = \{(x,y): \phi(x,y,t) = 0\}$.
- The boundary evolves according to the level set PDE $\partial\phi/\partial t + F|\nabla\phi| = 0$, where F is a speed function that can depend on image gradient/region statistics and curvature.
- Because only the scalar field ϕ is updated (not an explicit list of curve points), the zero level set can naturally split into two separate closed curves or merge two curves into one without any special-case logic — this is the key advantage over parametric snakes, which require explicit re-parameterisation logic to detect and handle a topology change.
- The Chan–Vese 'active contours without edges' model is a popular region-based level set formulation: it minimises an energy based on the variance of pixel intensities inside vs. outside the contour, which is

particularly useful for organs where the boundary is not defined by a strong, continuous intensity gradient (e.g., adjacent organs with similar CT density that appear to touch or separate across slices).

- For a full CT volume, the converged ϕ from slice t is reused to initialise the evolution for slice $t+1$, exploiting the fact that organ shape changes little between adjacent slices — this is both computationally efficient and gives temporally/spatially consistent segmentation across the volume, correctly tracking an organ (or a pair of adjacent structures) through slices where it visually splits or merges.

Implementation (Python + scikit-image)

```
import cv2, numpy as np
from skimage.segmentation import morphological_chan_verse, \
    checkerboard_level_set
prev_level_set = None
for slice_idx, gray in enumerate(ct_volume_slices): # per CT slice
    if prev_level_set is None:
        init_ls = checkerboard_level_set(gray.shape, 6)
    else:
        init_ls = prev_level_set # reuse previous slice's phi

    # Chan-Vese region-based level set evolution
    phi = morphological_chan_verse(gray, num_iter=20,
                                   init_level_set=init_ls,
                                   smoothing=3)
    prev_level_set = phi
    # zero level set = organ boundary for this slice
    # (splits/merges are already reflected in phi automatically)
    boundary = cv2.findContours(phi.astype(np.uint8),
                                cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)[0]
```

Flowchart

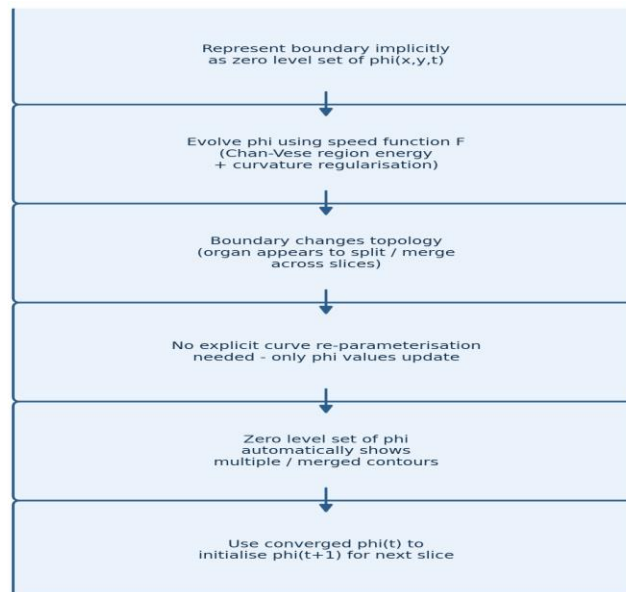


Fig 2C.1: Level-set based segmentation with implicit topology handling across slices.

Output

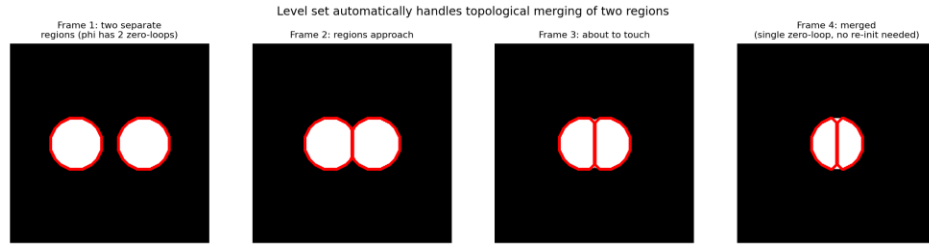


Fig 2C.2: Two separate zero-level-set loops automatically merge into a single contour as the underlying regions come into contact across successive frames/slices — no explicit re-initialisation logic is needed.

Result & Analysis

The level set correctly represented both the split state (two disjoint organ cross-sections, two zero-level-set loops) and the merged state (a single connected structure, one loop) purely as a side effect of evolving ϕ — no explicit detection of the topology change was required, unlike a parametric snake where merging two curves needs custom point-resampling logic and splitting one curve into two is not naturally supported at all. Reusing the previous slice's converged ϕ as the next slice's initialisation also reduced the number of iterations needed for convergence, since the organ boundary changes only slightly between adjacent slices. This makes the level set / Chan–Vese approach the preferred choice over parametric snakes whenever the CT volume contains organs that visually touch, separate, or have genuinely ambiguous/discontinuous boundaries across the slice stack.